

Projet C++ Atelier 3 : CRUD

Elaboré par :
Mme. Soumaya Argoubi

AU : 2020 -2021



Plan



Qt

Implémenter l'interface (GUI)

Qt

Créer une connexion à la Base de données (BD)

Qt

Créer la classe purement C++

Qt

Sécuriser l'application

Qt

Implémenter les fonctionnalités de base



► Objectif

- L'objectif de ce workshop est de manipuler la table **etudiant** via une interface graphique en y effectuant :
 - L'ajout d'un nouvel étudiant (*create*)
 - La suppression d'un étudiant (*delete*)
 - L'affichage de la liste des étudiants (*read*)

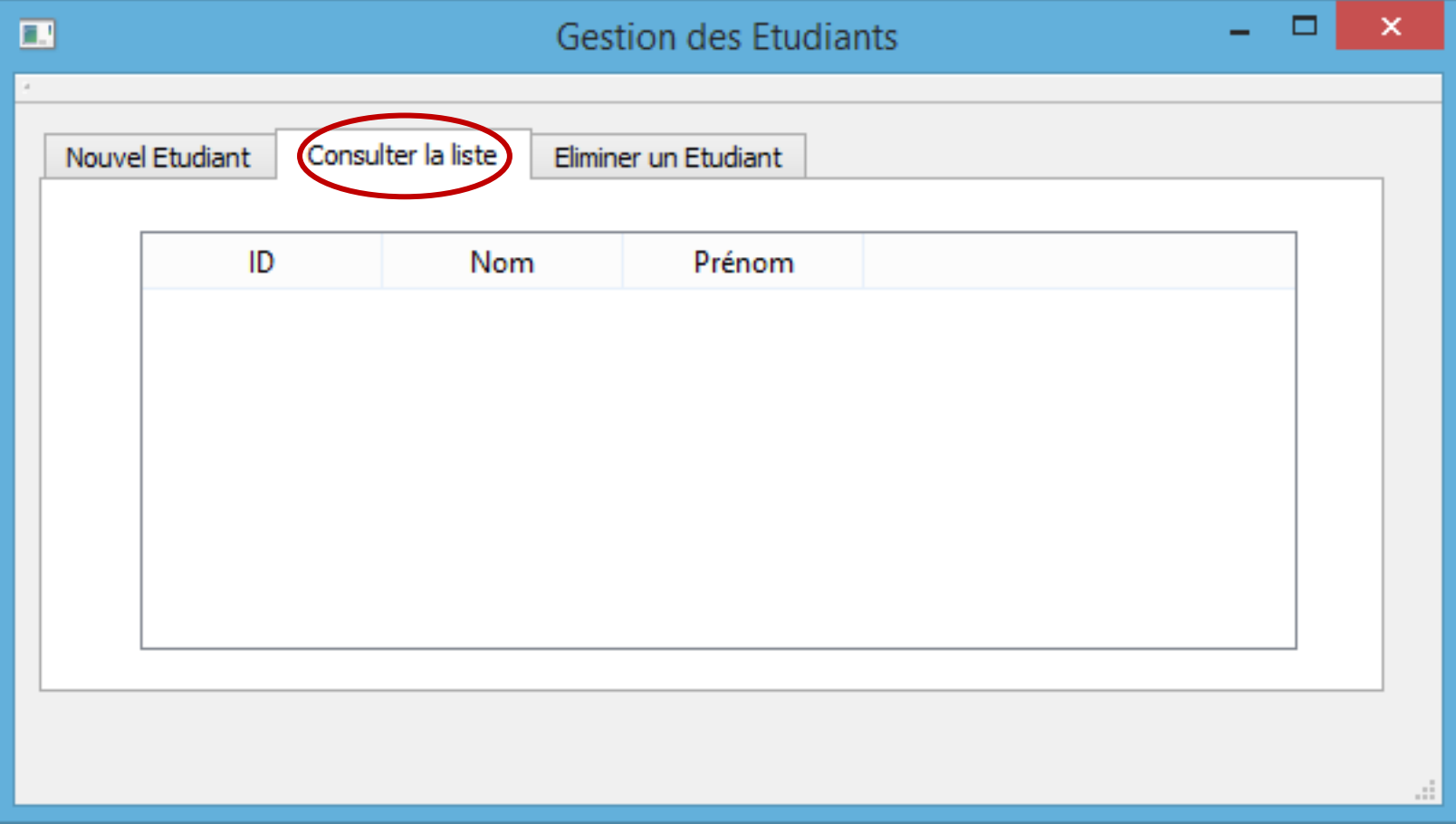
► Implémenter l'interface



► Implémenter l'interface « 1/3 »

The image shows a software window titled "Gestion des Etudiants". It features three tabs: "Nouvel Etudiant", "Consulter la liste", and "Eliminer un Etudiant". The "Nouvel Etudiant" tab is active, displaying three text input fields labeled "ID", "Nom", and "Prenom". To the right of these fields is a button labeled "Ajouter", which is highlighted with a red circle. The window has a standard Windows-style title bar with minimize, maximize, and close buttons.

► Implémenter l'interface « 2/3 »



Gestion des Etudiants

Nouvel Etudiant Consulter la liste Eliminer un Etudiant

ID	Nom	Prénom
----	-----	--------

► Implémenter l'interface « 3/3 »

The screenshot shows a Java Swing window titled "Gestion des Etudiants". At the top, there are three tabs: "Nouvel Etudiant", "Consulter la liste", and "Eliminer un Etudiant". The "Eliminer un Etudiant" tab is currently selected. Below the tabs, there is a form with a label "ID" followed by a text input field. Below the input field, there is a button labeled "Supprimer", which is circled in red. The window has a standard Mac OS X-style title bar with a red close button.



▶ Créer une Connexion à la BD



ORACLE®
DATABASE

► Créer une Connexion à la BD «1/3»

- Fichier « connection.h »

```
#ifndef CONNECTION_H
#define CONNECTION_H
#include <QSqlDatabase>

class Connection
{
    QSqlDatabase db;
public:
    Connection();
    bool createconnection();
    void closeConnection();
};

#endif // CONNECTION_H
```

db attribut de la classe
connection

► Créer une Connexion à la BD «2/3»

➤ Fichier « connection.cpp »

```
#include "connection.h"

Connection::Connection() {}

bool Connection::createconnection()
{
    db = QSqlDatabase::addDatabase("QODBC");
    bool test=false;
    db.setDatabaseName(""); //insérer le nom de la source de données ODBC
    db.setUserName(""); //insérer nom de l'utilisateur
    db.setPassword(""); //insérer mot de passe de cet utilisateur

    if (db.open()) test=true;

    return test;
}

void Connection::closeConnection() { db.close(); }
```

L'attribut db sera initialisé dans la méthode createconnection() qui établie la connexion à la BD



► Créer une Connexion à la BD «3/3»



- Vérifier que tout au long de votre projet ,vous allez manipuler **une seule instance** de la classe **Connection**

connection.h

```
#include "mainwindow.h"
#include <QApplication>
#include <QMessageBox>
#include "connection.h"

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    Connection c; //Une seule instance de la classe connection

    bool test=c.createconnection();//Etablir la connexion

    MainWindow w;

    if(test) //Si la connexion est établie
    {
        w.show();

        QMessageBox::information(nullptr, QObject::tr("database is open"),
            QObject::tr("connection successful.\n"
                "Click Cancel to exit."), QMessageBox::Cancel);

    }
    else //Si la connexion a échoué

        QMessageBox::critical(nullptr, QObject::tr("database is not open"),
            QObject::tr("connection failed.\n"
                "Click Cancel to exit."), QMessageBox::Cancel);

    return a.exec();
}
```

 **Créer la classe purement C++**





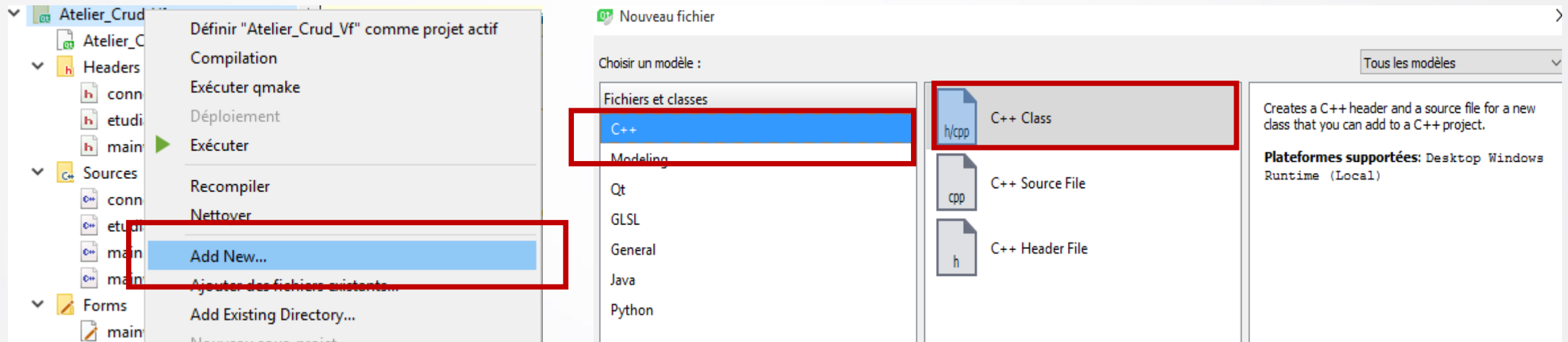
Créer la classe purement C++ « 1/4 »



Etudiant
-id -Nom -Prenom
+Etudiant() +Etudiant(int, string, string) +void afficher() +bool ajouter() +bool supprimer(int)

► Créer la classe purement C++ « 2/4 »

➤ Ajouter une nouvelle classe C++ « Etudiant » :



► Créer la classe purement C++ « 3/4 »

➤ Fichier « etudiant.h »

```
#ifndef ETUDIANT_H
#define ETUDIANT_H
#include<QString>
#include<QSqlQuery>
#include<QSqlQueryModel>

class Etudiant
{
    QString nom, prenom;
    int id;

public:
    //Constructeurs
    Etudiant(){}
    Etudiant(int,QString,QString);

    //Getters
    QString getNom(){return nom;}
    QString getPrenom(){return prenom;}
    int getID(){return id;}

    //Setters
    void setNom(QString n){nom=n;}
    void setPrenom(QString p){prenom=p;}
    void setID(int id){this->id=id;}

    // Fonctionnalités de Base relatives à l'entité Etudiant
    bool ajouter();
    QSqlQueryModel * afficher();
    bool supprimer(int);

};
```

► Créer la classe purement C++ « 4/4 »



➤ Fichier « etudiant.cpp »

```
#include "etudiant.h"

Etudiant::Etudiant(int id,QString nom,QString prenom)
{
    this->id=id;
    this->nom=nom;
    this->prenom=prenom;
}

bool Etudiant::ajouter()
{
    //ToDo
}

 QSqlQueryModel * Etudiant::afficher()
{
    //ToDo
}

bool Etudiant::supprimer(int id)
{
    //ToDo
}
```




Sécuriser l'application



► Sécuriser l'application



❖ Protéger la Base de données



- ❑ Nous allons manipuler la table Etudiant de notre BD via des **requêtes SQL**
- ❑ Il faut, donc, bien protéger notre BD.
- ❑ Exemple : [SQL injection](#), l'attaque de sécurité classée 1^{re} par l'OWASP
- ❑ Cette attaque consiste à utiliser ou « **injecter** » des expressions et les interpréter comme une partie de la requête.
- ❑ Exemple : Envoyer une requête dans les **champs** du **formulaire d'authentification** de l'application.

➔ **L'attaquant aura un accès libre à la BD**

► Sécuriser l'application



❖ Requête utilisée pour l'authentification

```
QString select = "SELECT login from account where login='"+  
    ui->lelogin->text()+"' and password='"+ui->lepassword->text()+"'";
```

```
QSqlQuery query;  
query.exec(select);
```

Exécution de la requête

► Sécuriser l'application



❖ Requête utilisée pour l'authentification

```
Connection DB OK  
"SELECT login from account where login='admin' and password='admin'"
```

1- Connection à la BD et
exécution de la requête

2- Identité vérifiée

MainWindow

login

password

Result: OK

3- Accès accordé

► Sécuriser l'application

❖ Exemple d'une attaque SQL injection



MainWindow

login admin

password toto' or '1'='1

let me in

Result:

Expression injectée : **toto ' or '1'='1**

❑ Requête envoyée à la BD : `QString select = "SELECT login from account where login='"+`
`ui->lelogin->text()+"' and password='"`
`ui->lepassword->text()+"`;

password = **toto' or '1'='1**

► Sécuriser l'application

❖ Exemple d'une attaque SQL injection



MainWindow

login

password

Result: OK

3- Accès accordé



1- Connection à la BD et
exécution de la requête

2- Identité vérifiée

Connection DB OK
"SELECT login from account where login='admin' and password='toto' **or** '1' = '1'"

► Sécuriser l'application

❖ Exemple d'une attaque SQL injection

3- Accès accordé COMMENT ???



Connection DB OK

"SELECT login from account where login='admin' and password='toto' **or** '1' = '1' "

→ La requête est : **SELECT login FROM account WHERE** login = 'admin' **AND** password = 'toto' **OR** '1'='1' ;

→ La condition du **WHERE** sera toujours évaluée à **True** car 1 est toujours égale à 1.

Condition du Where

→ La condition du **WHERE** est équivalente à : **X OR true → True**

→ La requête sera équivalente à : **SELECT login FROM account WHERE true;**

→ Cette requête est équivalente à : **SELECT login FROM account ; → accès à tout le contenu de la table account.**



► Sécuriser l'application

❖ Prévention :

- Eviter les requêtes simples.

```
QSqlQuery query;  
query.exec(select);
```
- Faire appel aux **requêtes préparées**.
 - ➔ Les paramètres sont interprétés indépendamment de la requête elle-même.

❖ Comment ?

- 1- La méthode **prepare()** : Prendra la requête en paramètre pour la préparer à l'exécution.
- 2- Dans la requête il faut utiliser des variables Bind (**BindValue**) et non pas les informations récupérées du formulaire.
- 3- La méthode **exec()** : Envoie la requête pour l'exécuter.



► Sécuriser l'application

❖ Requêtes préparées

- ✓ la requête est découpée en deux parties : **fixe** et **variable**.
 - **Partie fixe** : Sera envoyé à la base de données pour préparer l'exécution.
 - **Partie variable** : Va changer selon la valeur saisie dans le lineEdit. (Les paramètres qui seront liés (**binding**).)
- ➔ Les caractères spéciaux ne seront plus interprétés, expl : ‘
- ➔ **Protection contre les injections SQL**



► Sécuriser l'application

❖ Requêtes préparées

- ✓ Au lieu de mettre directement les valeurs, on va mettre des marqueurs (?) ou **un marqueur nominatif**) qui seront remplacés plus tard.
- ✓ Qt prend en charge deux syntaxes :
 - La liaison **nommée** (marqueur nominatif) : QSqlQuery::bindValue()
 - La liaison de **position** (?) : QSqlQuery::addBindValue()
- ✓ La requête sera réutilisable pour différentes valeurs → Optimisation des requêtes SQL
- ✓ On n'écrit pas la requête dans la méthode QSqlQuery::exec(), mais dans la méthode QSqlQuery::prepare().

► Sécuriser l'application

❖ Requêtes préparées : QSqlQuery::addBindValue()

```
QSqlQuery query(db);  
QString select = "SELECT login from account where login=? and password=?";  
qDebug() << select;  
query.prepare(select);  
query.addBindValue(ui->lelogin->text());  
query.addBindValue(ui->lepassword->text());  
query.exec();
```

► Sécuriser l'application

❖ Requêtes préparées : QSqlQuery::bindValue()

```
QSqlQuery query;

QString res = QString::number(id);

//prepare() prend la requête en paramètre pour la préparer à l'exécution.

query.prepare("insert into etudiant (id, nom, prenom)" "values (:id, :nom, :prenom)");

//Création des variables liées
query.bindValue(":id",res);
query.bindValue(":nom",nom);
query.bindValue(":prenom",prenom);

query.exec(); //exec() envoie la requête pour l'exécuter
```

BindValue (variable précédée par « : » et insérée dans la partie SQL)

Associer une variable applicatives (contenant les valeurs saisies dans les lineEdits) à chaque bindvalue

La requête n'est plus passée à exec() (Comme [diapo 19](#)), mais à la méthode prepare() .



► Implémenter les fonctionnalités de base



ORACLE®
DATABASE

▶ AJOUT



❖ Implémentation de la méthode ajouter() :

etudiant.cpp

```
bool Etudiant::ajouter()
{
    QSqlQuery query;

    QString res = QString::number(id);

    //prepare() prend la requête en paramètre pour la préparer à l'exécution.

    query.prepare("insert into etudiant (id, nom, prenom)" "values (:id, :nom, :prenom)");

    //Création des variables liées
    query.bindValue(":id",res);
    query.bindValue(":nom",nom);
    query.bindValue(":prenom",prenom);

    return query.exec(); //exec() envoie la requête pour l'exécuter
}
```


▶ AJOUT



❖ Appel de la méthode ajouter() :

Aucune requête SQL et aucun accès à la base de données ne doit se faire en dehors de la classe C++ .



```
void MainWindow::on_pushButton_ajouter_clicked()
```

mainwindow.cpp

```
{  
    //Récupération des informations saisies dans les 3 champs  
    int id=ui->lineEdit_ID->text().toInt();  
    QString nom=ui->lineEdit_nom->text();  
    QString prenom=ui->lineEdit_prenom->text();
```

Convertir la chaîne saisie en un entier car l'attribut id est de type int

```
    Etudiant E(id,nom,prenom); // instancier un objet de la classe étudiant  
                                // en utilisant les informations saisies dans l'interface  
  
    bool test=E.ajouter(); // Insérer l'objet étudiant instancié dans la table étudiant  
                           // et récupérer la valeur de retour de query.exec()
```

```
    if(test) // Si requête exécutée ==> QMessageBox::information  
    {  
        QMessageBox::information(nullptr, QObject::tr("OK"),  
                                QObject::tr("Ajout effectué\n" "Click Cancel to exit."), QMessageBox::Cancel);  
    }  
    else // Si requête non exécutée ==> QMessageBox::critical  
        QMessageBox::critical(nullptr, QObject::tr("Not OK"),  
                               QObject::tr("Ajout non effectué.\n" "Click Cancel to exit."), QMessageBox::Cancel);  
}
```

▶ AJOUT



❖ Exécution :

The screenshot displays a Windows application titled "Gestion des Etudiants". It features three tabs: "Nouvel Etudiant" (selected), "Consulter la liste", and "Eliminer un Etudiant". Under the "Nouvel Etudiant" tab, there are three input fields: "ID" with the value "12345", "Nom" with the value "User", and "Prenom" with the value "ForTest". An "Ajouter" button is located at the bottom right of the form. An information dialog box is overlaid on the right side of the application, titled "OK", with a blue information icon. The dialog contains the text "Ajout effectué" and "Click Cancel to exit.", with a "Cancel" button at the bottom.



► Suppression

❖ Implémentation de la méthode supprimer() :

etudiant.cpp

```
bool Etudiant::supprimer(int id)
{
    QSqlQuery query;
    QString res=QString::number(id);

    query.prepare("Delete from etudiant where ID= :id");

    query.bindValue(":id",res);

    return query.exec();
}
```

► Suppression



❖ Appel de la méthode supprimer() :

mainwindow.h

```
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void on_pushButton_ajouter_clicked();

private:
    Ui::MainWindow *ui;
    Etudiant Etmp;
};
```

Ajouter un attribut à la classe mainwindow qui correspond à un objet de la classe étudiant pour pouvoir faire appel à la méthode supprimer ()

► Suppression



❖ Appel de la méthode supprimer() :

mainwindow.cpp

```
void MainWindow::on_pushButton_supprimer_clicked()
{
    int id = ui->lineEdit_IDS->text().toInt();
    bool test = Etmp.supprimer(id);
    if(test)
    {
        QMessageBox::information(nullptr, QObject::tr("OK"),
            QObject::tr("Suppression effectuée\n"
                "Click Cancel to exit."), QMessageBox::Cancel);
    }
    else
        QMessageBox::critical(nullptr, QObject::tr("Not OK"),
            QObject::tr("Suppression non effectuée.\n"
                "Click Cancel to exit."), QMessageBox::Cancel);
}
```

Convertir la chaîne saisie en un entier
car l'attribut id est de type int

Appel de la méthode supprimer() via l'attribut Etmp



Suppression



❖ Exécution :

The screenshot shows a software application window titled "Gestion des Etudiants". At the top, there are three tabs: "Nouvel Etudiant", "Consulter la liste", and "Eliminer un Etudiant". The "Eliminer un Etudiant" tab is currently selected. Below the tabs, there is a form with a label "ID" and a text input field containing the number "12". To the right of the input field is a button labeled "Supprimer". Overlaid on the right side of the application window is a smaller dialog box titled "OK". This dialog box contains an information icon (a blue circle with a white 'i') and the text "Suppression effectuée Click Cancel to exit." At the bottom of the dialog box is a button labeled "Cancel".

► AFFICHAGE



❖ Implémentation de la méthode afficher() :

etudiant.cpp

```
 QSqlQueryModel * Etudiant::afficher()
{
    QSqlQueryModel * model=new QSqlQueryModel();

    model->setQuery("select * from etudiant");
    model->setHeaderData(0,Qt::Horizontal,QObject::tr("ID"));
    model->setHeaderData(1,Qt::Horizontal,QObject::tr("Nom"));
    model->setHeaderData(2,Qt::Horizontal,QObject::tr("Prénom"));

    return model;
}
```

► AFFICHAGE



❖ Appel de la méthode afficher() :

Constructeur de la classe
mainwindow généré
automatiquement

mainwindow.cpp

```
MainWindow::MainWindow(QWidget *parent) :  
    QMainWindow(parent),  
    ui(new Ui::MainWindow)  
{  
    ui->setupUi(this);  
    ui->tableView->setModel(Etmp.afficher());  
}
```

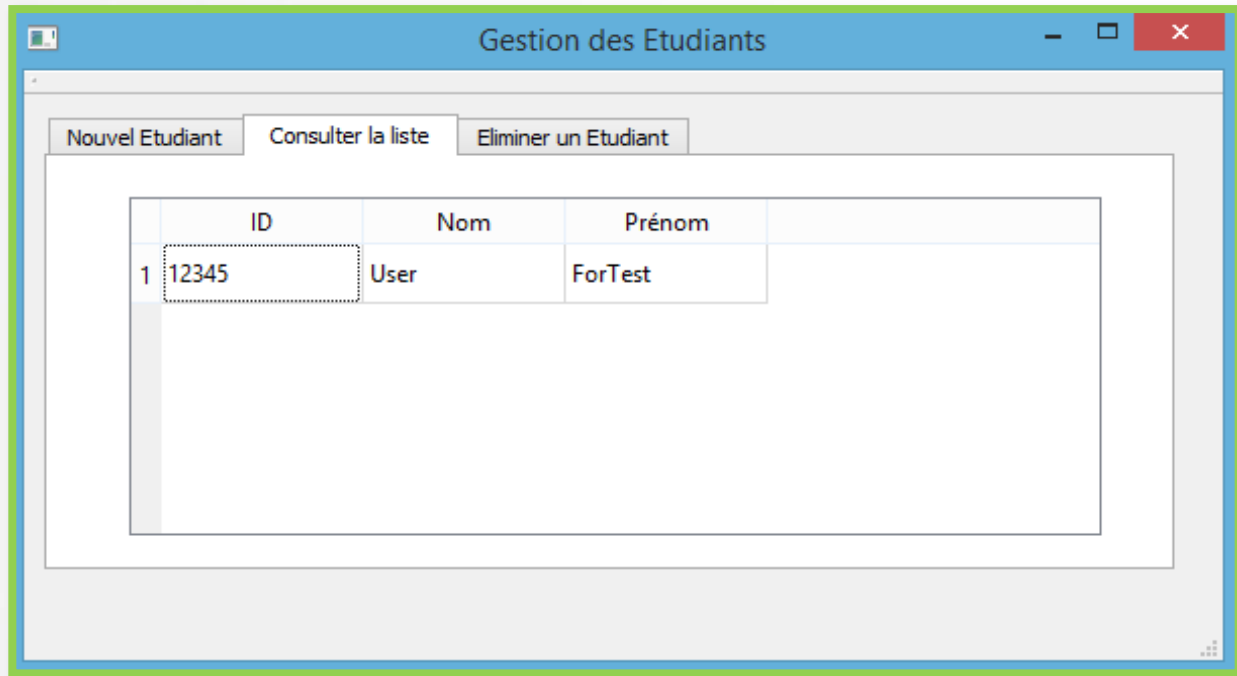
Instruction ajoutée dans le constructeur : Appel de la
méthode afficher () via l'attribut Etmp
(voir [diapo 34](#))



AFFICHAGE



❖ Exécution : Affichage du contenu de la table lors de l'exécution de l'application



```
Connection c;
```

main.cpp

```
bool test=c.createconnection();//Etablir la connexion
```

```
MainWindow w;
```

Appel du constructeur →
appel de la méthode afficher()

L'instanciation de l'objet de la classe mainwindow dans le main doit se faire **après avoir établie la connexion à la BD** pour avoir l'affichage lors de l'exécution car l'appel de la méthode afficher() se fait dans le constructeur de la classe mainwindow et l'appel du constructeur se fait lors de l'instanciation de l'objet W.



Actualisation de l'affichage



- ❖ Maintenant, si on essaye de faire l'affichage de la liste des étudiants après un ajout ou bien une suppression, nous aura la même liste sans mise à jour.
- ❖ **Pourquoi ?** Car la requête **select** est exécutée dans la méthode **afficher()** qui est appelée uniquement dans **le constructeur** de la classe mainwindow !
- ➔ L'affichage du contenu de la table **etudiant** ne se fait **qu'une seule fois** : lors de l'exécution (**appel du constructeur** de la classe mainwindow pour instancier l'objet w)
- ➔ **Solution** : la requête **select** doit être exécutée **après chaque requête** qui peut **modifier** le contenu de la table **etudiant**.



Actualisation de l'affichage



❖ Actualiser après ajout :

mainwindow.cpp

```
void MainWindow::on_pushButton_ajouter_clicked()
{
    int id=ui->lineEdit_ID->text().toInt();
    QString nom=ui->lineEdit_nom->text();
    QString prenom=ui->lineEdit_prenom->text();

    Etudiant E(id,nom,prenom);

    bool test=E.ajouter();

    if(test) // Si requête exécutée
    {
        // Refresh (Actualiser)
        ui->tableView->setModel(Etmp.afficher());

        QMessageBox::information(nullptr, QObject::tr("OK"),
                                   QObject::tr("Ajout effectué\n"
                                                "Click Cancel to exit."), QMessageBox::Cancel);
    }
    else // Si requête non exécutée
        QMessageBox::critical(nullptr, QObject::tr("Not OK"),
                               QObject::tr("Ajout non effectué.\n"
                                            "Click Cancel to exit."), QMessageBox::Cancel);
}
```

Appeler la méthode afficher() → exécuter la requête select si l'ajout a eu lieu (if (test)).



Actualisation de l'affichage



❖ Actualiser après suppression:

mainwindow.cpp

```
void MainWindow::on_pushButton_supprimer_clicked()
{
    int id = ui->lineEdit_IDS->text().toInt();
    bool test = Etmp.supprimer(id);

    if(test)
    {
        // Refresh (Actualiser)
        ui->tableView->setModel(Etmp.afficher());

        QMessageBox::information(nullptr, QObject::tr("OK"),
                                QObject::tr("Suppression effectuée\n"
                                                "Click Cancel to exit."), QMessageBox::Cancel);
    }
    else
        QMessageBox::critical(nullptr, QObject::tr("Not OK"),
                              QObject::tr("Suppression non effectuée.\n"
                                              "Click Cancel to exit."), QMessageBox::Cancel);
}
```

Appeler la méthode afficher() → exécuter la requête select si la suppression a eu lieu (if (test)).



Récapitulation



❖ **CRUD** : Fonctionnalités de base

- ✓ **C**reate : Ajouter 
- ✓ **R**ead : Afficher 
- ✓ **U**pdate : Modifier 
- ✓ **D**elelete : Supprimer 

❖ **Métiers & Arduino** : fonctionnalités avancées



Consignes et remarques



- ❖ Une seule connexion à la BD.
- ❖ Requêtes **préparées**.
- ❖ L'interface graphique proposée n'est qu'un exemple **simple** pour comprendre le **principe**.
- ❖ Essayez d'**innover** en proposant des GUI **ergonomiques**.
- ❖ **Optimiser** l'espace : expl : l'interface de la suppression ici contient uniquement 3 composants : label, lineEdit et bouton ! ➔ à éviter.
- ❖ Opter plutôt pour les « containers », tels que : **Tab Widget, Stacked widget**.

Consignes et remarques



- ❖ **Minimiser** les « dialog » au maximum.
- ❖ Contrôle de saisie obligatoire dans le code C++.
- ❖ Mettre les contraintes nécessaires dans les tables au niveau de la BD.
- ❖ Les requêtes doivent être implémentées dans les classes purement **C++ relatives aux entités gérées par l'application.**
- ❖ **Aucune** requête ne doit être implémentée derrière un bouton.



Références



https://owasp.org/www-community/attacks/SQL_Injection

<https://qt-quarterly.developpez.com/qq-04/qt-fonctionnalites-sql/>

<https://alain-defrance.developpez.com/articles/Qt/SGBD/>



BON TRAVAIL

La réussite appartient à tout le monde. C'est au travail d'équipe qu'en revient le mérite.

Franck Piccard